

Курсова Работа

Операционни системи

Вариант 1

1 Въведение

За този вариант на курсовата работа, ще трябва да напишете няколко малки програми, всяка решение на дадена задача. Всички задачи се решават на езика C.

2 Задачи

2.1 Hex dump

Напишете програма `hexdump`, която получава път към файл като аргумент от командния ред и изписва съдържанието на файла в шестнадесетичен формат на стандартния изход. Трябва да се изнесат само по 16 байта на ред. За всеки байт се извеждат 2 знака (0-9, A-F). Между байтовете се извежда по един знак за разстояние, освен между 8. и 9. байт, между тях са 2 знака, за да се формират 2 групи по 8 байта. След шестнадесетичното представяне, с 3 знака за разстояние, се извежда целия ред като ASCII знаци. Ако байта е извън обхвата на знаците, които могат да се изпишат (32-127 вкл.), вместо знака се извежда ".". Ако размера на файла не е кратен на 16, шестнадесетичното представяне на файла приключва по-рано, но ASCII представянето трябва да е на същото място като на другите редове (липсващите байтове се заменят с разстояния). В случай на грешки (липсващ файл, няма права, т.н.), да се извеждат на стандартния изход за грешки с префикса `hexdump` и програмата да приключва с код 1. В кода си не може да използвате функции от `stdio.h` с изключение на `perror`.

Примери:

```
$ cat file.txt
Hello, world!
$ ./hexdump file.txt
48 65 6C 6C 6F 20 77 6F 72 6C 6D 21                Hello, world!...

$ cat file.txt
Some longer text that should end up on multiple lines:)
$ ./hexdump file.txt
53 6F 6D 65 20 6C 6F 6E 67 65 72 20 74 65 78 74    Some longer text
20 74 68 61 74 20 73 68 6F 75 6C 64 20 65 6E 64    that should end
20 75 70 20 6F 6E 20 6D 75 6C 74 69 70 6C 65 20    up on multiple
6C 69 6E 65 73 20 3A 29                            lines:).....

$ ./hexdump file.txt
54 68 69 73 20 69 73 20 61 20 62 65 6C 6C 07      This is a bell..
```

```
$ ./hexdump_this-file-does-not-exist
hexdump: No such file or directory
$ echo $?
1
```

```
$ ./hexdump_protected-file
hexdump: Permission denied
$ echo $?
1
```

2.2 Обръщане на файл на място

Напишете програма **reverse**, която получава път към файл като единствен аргумент от командния ред и обръща байтовия ред на файла *на място*, т.е. без да зарежда целия файл в паметта наведнъж. Програмата трябва да работи с буфер с фиксиран размер от точно **512 байта**. Разрешено е да използвате само системните извиквания **open**, **read**, **write**, **lseek** и **close**; не може да използвате функции от **stdio.h** с изключение на **perror** и не можете да използвате динамично заделяне на памет (**malloc**, **mmap**, **free**, ...)

В случай на грешка (липсващ файл, няма права за запис, и т.н.) програмата извежда диагностика на стандартния изход за грешки с префикс **reverse** и завършва с код 1.

Примери:

```
$ printf 'abcdef' > file.bin
$ ./reverse file.bin
$ cat file.bin
fedcba
```

```
$ printf 'Hello, world!' > file.bin
$ ./reverse file.bin
$ cat file.bin
!dlrow, olleH
```

```
$ ./reverse_no-such-file
reverse: No such file or directory
$ echo $?
1
```

2.3 Паралелно изпълнение на команди

Напишете програма **prun**, която получава произволен брой командни низове като аргументи от командния ред (всеки аргумент е цяла команда с аргументи, разделени с интервали) и ги изпълнява *едновременно* – за всяка команда се създава отделен дъщерен процес чрез **fork**, след което дъщерният процес изпълнява командата чрез **execvp**. Бащиният процес изчаква всички дъщерни процеси да завършат с **waitpid** и за всеки от тях извежда на стандартния изход един ред в следния формат: [**<pid>**] "**<команда>**" **exited with status <код>** Редовете се извеждат в реда, в който процесите завършват (т.е. в реда, в който **waitpid** ги прибира). В случай на грешка при **fork** или **execvp** програмата извежда диагностика на стандартния изход за грешки с префикс **prun**. Не може да използвате **system(3)** или **rope(3)**.

Примери:

```
$./prun "echo hello" "sleep 0.1" "ls /nonexistent"
[12345] "ls /nonexistent" exited with status 2
[12344] "echo hello" exited with status 0
[12346] "sleep 0.1" exited with status 0
```

2.4 Мини-шел с тръба

Напишете програма `pipe2`, която получава точно два аргумента – низове, представлящи по една shell-команда с аргументи разделени с интервали – и изпълнява `cmd1 | cmd2`: стандартният изход на `cmd1` се свързва с стандартния вход на `cmd2` чрез анонимна тръба, създадена с `pipe(2)`. Двете команди се изпълняват *едновременно* (не последователно) в отделни дъщерни процеса, създадени с `fork` и изпълнени с `execvp`. Бащиният процес затваря двата края на тръбата след като ги е предал на децата, изчаква и двете деца с `waitpid`, и излиза с кода за изход на *дясната* команда (`cmd2`). Ако някое дете е убито от сигнал, кодът за изход на бащата е `128 + номер_на_сигнала`. В случай на грешка при `pipe`, `fork` или `execvp` програмата извежда диагностика на стандартния изход за грешки с префикс `pipe2`. Не може да използвате `system(3)`, `popen(3)` или именувани тръби.

Примери:

```
$./pipe2 "ls /usr/bin" "wc -l"
42
$./pipe2 "echo $"
0

$./pipe2 "echo hello world" "tr a-z A-Z"
HELLO WORLD

$./pipe2 "cat /dev/urandom" "head -c 4" | xxd
00000000: e3a1f20b.....
```

Подсказка: ще ви е нужна функцията `dup2`

2.5 Изящно спиране

Напишете програма `worker`, която изпълнява повтаряща се работа в безкраен цикъл: веднъж в секунда (използвайки `sleep(1)`) извежда на стандартния изход ред в следния формат:

```
tick <брояч>
```

където `брояч` е монотонно нарастващо цяло число, започващо от 1.

Програмата трябва да обработва следните сигнали чрез `sigaction`:

- `SIGINT` и `SIGTERM`: задават флаг от тип `volatile sig_atomic_t`, с което казват на главния цикъл да спре. При излизане програмата извежда на стандартния изход ред `shutdown after <брояч> ticks` и завършва с код 0.
- `SIGUSR1`: извежда на стандартния изход ред `status: tick <брояч>` с текущата стойност на брояча, без да прекъсва нормалния ход на изпълнение.

Не трябва да се използват `signal(2)`, `sigsetjmp/siglongjmp` или нишки. Флаговете за сигнали трябва да бъдат декларирани като `volatile sig_atomic_t`.

Примери (стандартен изход):

```

$_./worker_&
PID=$!
sleep_2.5
kill_-$USR1_$PID
sleep_1
kill_-$INT_$PID
wait_$PID

tick_1
tick_2
status:_tick_2
tick_3
shutdown_after_3_ticks

```

2.6 Времеви сървър

Напишете *два* изпълними файла: `timeserver` и `timeclient`.

Сървърът получава порт като единствен аргумент от командния ред, свързва TCP сокет към `0.0.0.0:<порт>` с опция `SO_REUSEADDR`, слуша за входящи връзки, приема *една* връзка, извежда текущото местно време на клиента като форматиран низ (формат `YYYY-MM-DD HH:MM:SS\n`, чрез `strftime`), затваря връзката и излиза с код 0.

Клиентът получава `hostname` и порт като аргументи от командния ред, свързва се към сървъра, чете отговора (до затваряне на връзката), извежда го на стандартния изход и излиза с код 0.

В случай на грешка (невалиден порт, неуспешен `bind`, неуспешно `connect`, и т.н.) съответният изпълним файл извежда диагностика на стандартния изход за грешки с префикс `timeserver` или `timeclient` съответно, и завършва с код 1.

Примери:

```

$_./timeserver_12345_&
$_./timeclient_127.0.0.1_12345
2025-06-01_14:23:07

$_./timeclient_127.0.0.1_9
timeclient:_Connection_refused
$_echo_$?
1

```

2.7 Ехо-сървър с конкурентни клиенти

Напишете *два* изпълними файла: `echoserver` и `echoclient`.

Сървърът получава порт като единствен аргумент, свързва TCP сокет с `SO_REUSEADDR`, и влиза в безкраен цикъл на приемане на връзки (`accept`). За всяка приета връзка сървърът създава *нов процес* с `fork` (или нова нишка с `pthread_create` – изборът е ваш). Обработчикът чете данни от клиента на блокове от 1024 байта и ги изпраща обратно дословно, докато клиентът не затвори връзката. След това обработчикът затваря сокета и излиза. Бащиният процес (или главната нишка) не трябва да блокира в обработчика – той трябва веднага да се върне към `accept`. В случай на сигнал `SIGCHLD` (ако е избран вариантът с `fork`), бащата трябва да прибере завършилите деца с неблокиращ `waitpid(-1, ..., WNOHANG)`, за да не се натрупват зомби-процеси.

Клиентът получава hostname, порт и **низ за изпращане** като аргументи от командния ред. Свързва се към сървъра, изпраща низа, чете ехото и го извежда на стандартния изход, след което затваря връзката и излиза с код 0.

Примери:

```
$ ./echoserver 12345 &
$ ./echoclient 127.0.0.1 12345 "hello"
hello
$ ./echoclient 127.0.0.1 12345 "world"
world

$ for i in 1 2 3; do ./echoclient 127.0.0.1 12345 "msg$i" & done
msg2
msg3
msg1
```

3 Материали за предаване

Трябва да предадете git хранилище с изходния Ви код, публично достъпно в GitHub. В основната директория на хранилището трябва да има Makefile, в който има дефинирани target-и за всяка една програма от задачите. Името на всеки от тези target-и трябва да е името на програмата от задачата и да създава изпълнимия файл в главната директория (root-a) на проекта. Например, target-a за първата задача трябва да се казва **hexdump** и да прави файл **./hexdump**.

Също трябва да предадете PDF документ, в който да опишете как всяко ваше решение работи. Описанията са в свободен текст, но могат да включват и диаграми.

4 Оценяване

Критерий	Тежест
Функционалност	55%
Документация	45%

Функционалност ще се оценява на основа автоматични тестове ($\frac{\text{брой успешно минати тестове}}{\text{общ брой тестове}}$). За ваше удобство ще е предоставена извадка от тези тестове, които да използвате за да си потвърдите работата.

Ще получите оценка за всеки отделен критерии, които оценки после ще се съберат в една спрямо тежестите в таблицата. За закъснение се отнемат по 10% от оценката за всеки ден. Крайните проценти се привеждат в оценки по следната скала:

Процент	Оценка
0-44.99%	Слаб 2
45-54.99%	Среден 3
55-64.99%	Добър 4
65-74.99%	Много добър 5
75-100%	Отличен 6